

BFCL Jetson Test v2 — Benchmark Report

Which small LLMs can actually do things — call tools, chain steps, and write working code — on a single 8GB board? This benchmark takes the 17 best function-callers from BFCL Jetson Test v1 and puts them through a harder exam: 50 tool-calling tests plus two auto-graded coding builds, all served locally by llama.cpp on a Jetson Orin Nano 8GB. Every model runs in the same hardware envelope you would actually deploy — no cloud, no GPU cluster, just one small board doing its best.

The headline: single-call accuracy is nearly saturated at this scale — the real separation comes from chained multi-step calls and code that actually runs. A 3B Granite model topping the leaderboard at 96.2% is the kind of result that makes edge AI feel less like a compromise and more like a choice.

Companion repo: <https://github.com/drwjirkpatrick-web/bfcl-jetson-test-v2> — harness code, full per-test JSON results, and settings snapshot. Full source list on page 6. Run date 2026-09-29.

1. Test Suite

Four categories, 52 tests, all auto-graded. Evaluation is deterministic and executable — no LLM judge, no fluctuation. Tool-call matching follows BFCL-style rules: strict on required parameters, lenient on extras, string normalization, int/float coercion, order-independent matching for parallel calls. A chained test passes only if every step matches.

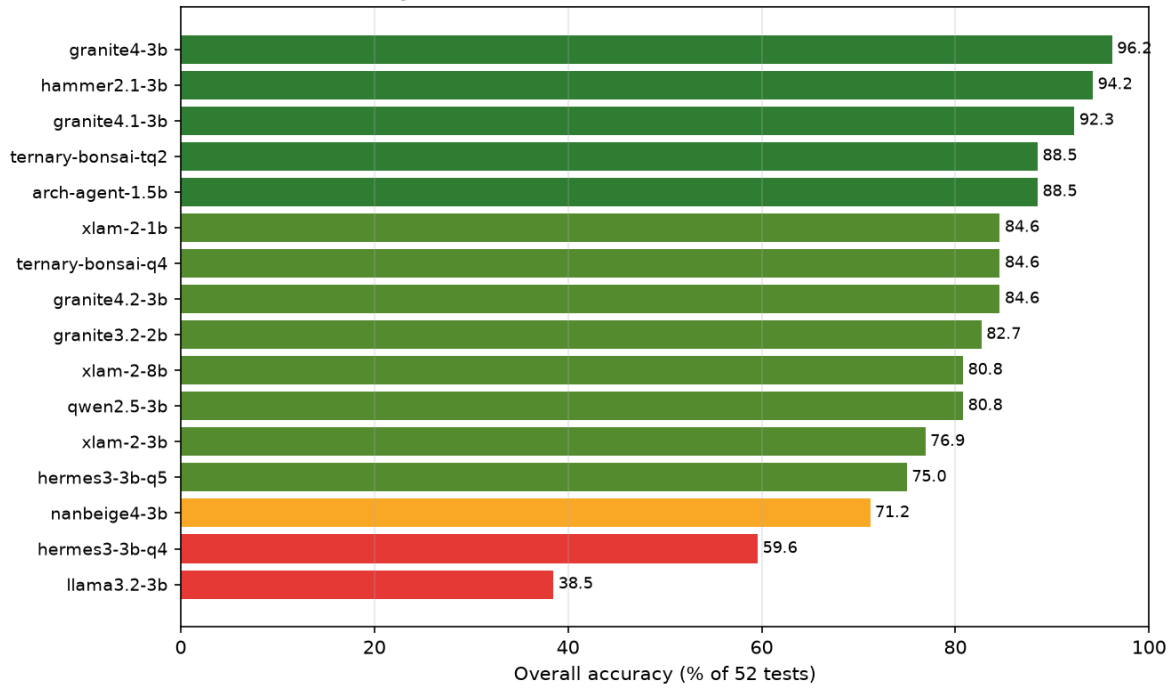
Category	Count	What it measures
simple	30	Single call with tricky argument extraction: ISO 8601 dates, 24-hour times, currency codes, booleans, arrays, unit conversion ("18% tip on an \$85.40 bill" to <code>calculate_tip(bill_amount=85.40, tip_percentage=18)</code>)
moderate_parallel	12	2-3 independent calls in one response, matched order-independently (BFCL "parallel" style)
moderate_chained	8	The divider. The harness executes step 1 with a deterministic mock, feeds the result back as a tool message, and the model must derive step-2 arguments from the returned data — including conditionals ("if below zero, set the thermostat")
python_game (coding)	1	Terminal tic-tac-toe: human vs computer, input validation, win/tie detection, play-again loop, stdlib only. Grade: 4 checks (board display, input validation, win detection, executes cleanly with piped input); pass = 3/4
html_profile (coding)	1	single-file profile page for a fictional photographer: hero, skills, contact form, dark theme, @media query, JS form validation. Grade: 7 structural checks; pass = 5/7

Extraction uses two paths, mirroring BFCL's separate FC and Prompt tracks. Path 1 reads the structured `tool_calls` field produced by llama.cpp's autoparser when the template renders tool markers. Path 2 is a text fallback for models whose GGUF templates never trigger the grammar: it scans content for JSON call emissions (arrays, fenced JSON, bare objects), tolerates arguments/parameters/args keys, and dedupes exact copies. Without path 2, a prompt-mode model scores near zero on structured-only reading — proven live when granite3.2-2b jumped from a false 1.9% to 82.7% once the fallback landed.

2. Results

16 of 17 models complete; qwen3.5-9b paused after two kernel OOM kills (details on page 4). All scores from the fixed v2.1 harness: structured extraction with text fallback, duplicate-call dedupe, per-model memory sizing.

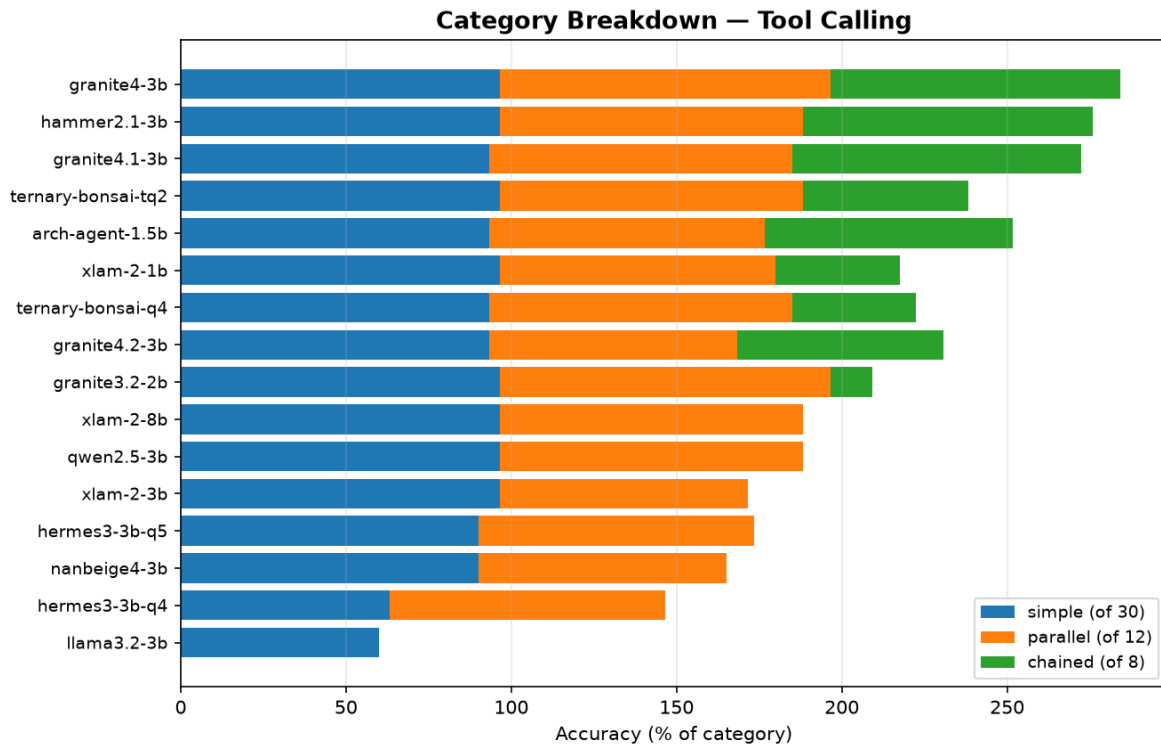
BFCL Jetson Test v2 — Overall Results (16 models)



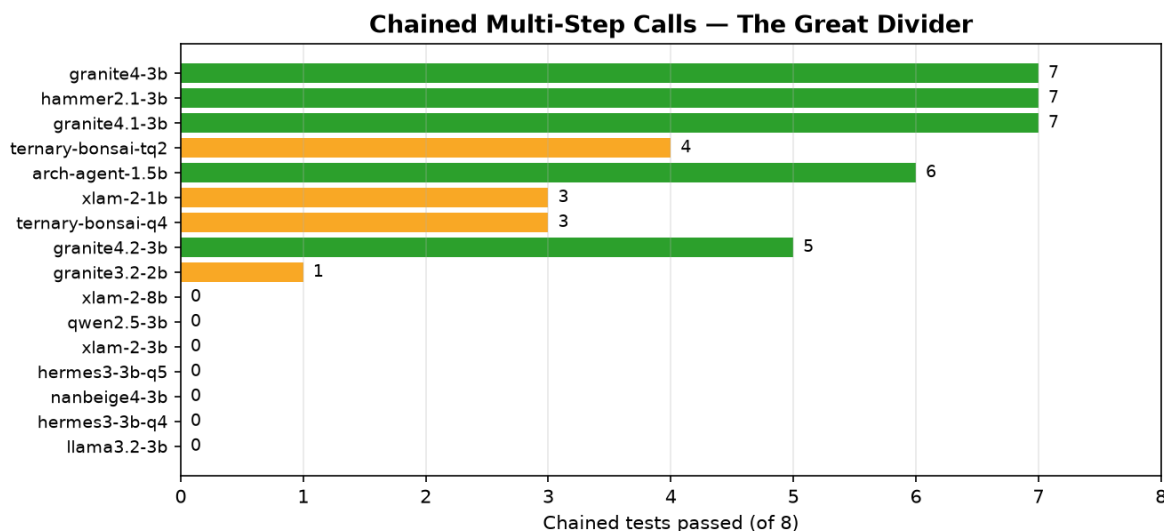
Full Results Table

Model	Overall	Simple	Par.	Chain.	Code	Notes
granite4-3b	50/52 (96.2%)	29/30	12/12	7/8	2/2	champion — perfect parallel
hammer2.1-3b	49/52 (94.2%)	29/30	11/12	7/8	2/2	biggest v1-to-v2 climb
granite4.1-3b	48/52 (92.3%)	28/30	11/12	7/8	2/2	
ternary-bonsai-tq2	46/52 (88.5%)	29/30	11/12	4/8	2/2	ternary, sub-4-bit — and it codes
arch-agent-1.5b	46/52 (88.5%)	28/30	10/12	6/8	2/2	best small agentic profile
xlam-2-1b	44/52 (84.6%)	29/30	10/12	3/8	2/2	1.6GB file — efficiency star
ternary-bonsai-q4	44/52 (84.6%)	28/30	11/12	3/8	2/2	
granite4.2-3b	44/52 (84.6%)	28/30	9/12	5/8	2/2	
granite3.2-2b	43/52 (82.7%)	29/30	12/12	1/8	1/2	text-fallback rescue (1.9% to 82.7%)
xlam-2-8b	42/52 (80.8%)	29/30	11/12	0/8	2/2	
qwen2.5-3b	42/52 (80.8%)	29/30	11/12	0/8	2/2	
xlam-2-3b	40/52 (76.9%)	29/30	9/12	0/8	2/2	
hermes3-3b-q5	39/52 (75.0%)	27/30	10/12	0/8	2/2	
nanbeige4-3b	37/52 (71.2%)	27/30	9/12	0/8	1/2	thinking model
hermes3-3b-q4	31/52 (59.6%)	19/30	10/12	0/8	2/2	Q4 hurts simple, not parallel
llama3.2-3b	20/52 (38.5%)	18/30	0/12	0/8	2/2	codes fine, calls poorly

Coding scores are pass/fail per test at the thresholds in section 1; overall treats each coding test as one point of 52.



3. What the Numbers Say



Chained calls are the great divider. Granite 4.x and Hammer2.1 pass 5-7 of 8 chained tests; nine models score exactly 0 — they emit a correct first call, then cannot derive second-step arguments from returned data. This is BFCL's multi-turn gap reproduced at edge scale.

Simple is saturated. Tool-trained models cluster at 27-29/30 — consistent with BFCL's single-turn saturation, which is exactly why BFCL re-weighted toward agentic categories.

4. Harness Honesty and Memory Limits

Two 0% scores during the run were harness artifacts, not model failures — fixed by the text fallback and duplicate-call dedupe, then retested (granite3.2-2b: 1.9% to 82.7%; xlam-2-1b: 3.8% to 84.6%). A benchmark that cannot tell those apart is not measuring models, it is measuring itself.

Coding is table stakes — with two exceptions. Fifteen of sixteen models built a runnable tic-tac-toe and a structurally sound HTML page. granite3.2-2b passed Python but produced no parsable HTML, and llama3.2-3b codes 2/2 while calling tools at 38.5%. Quantization bites unevenly: Hermes3-3B Q4 loses 8 simple-call points to Q5 yet matches it on parallel. Ternary Bonsai TQ2_0 beats its own Q4_0 on chained tests (4/8 vs 3/8).

qwen3.5-9b (4.1GB DeltaNet hybrid) was OOM-killed twice by the kernel — the same model that the v1 screen had passed at 50%. Server launch configs for the big models: 24K context with q8_0 KV failed twice, so models above 4.5GB now run 16K context with q4_0 KV and a 12,288-token coding cap; on a shared 8GB pool, KV quantization is what lets a 4.9GB model finish at all.

Also documented: two server-start allocation failures before the big-model rule existed. Next attempt for qwen3.5-9b: 12K context, q4_0 KV, 8,192-token coding cap.

5. Hardware and Server

Target	NVIDIA Jetson Orin Nano 8GB, ARM64, CUDA; GUI (GDM) stopped during runs to free ~6GB RAM
Server	llama.cpp llama-server --jinja, OpenAI-compatible API, GGML_CUDA_ENABLE_UNIFIED_MEMORY=1, flash attention on; autoparser analyzes chat templates to parse tool calls via a differential approach inspired by the git diff algorithm
Context	24576 tokens, q8_0 KV for models up to ~4.5GB; 16384 tokens, q4_0 KV, 12,288-token coding cap above
Sampling	temperature 0.0, top-k 1, repeat penalty 1.0 for tool tests (deterministic); temperature 0.7 for coding
Scoring	Tool: correct/50. Coding: pass/fail per test. Overall: correct/52, categories reported separately

6. Sources

[1]	gorilla.cs.berkeley.edu/leaderboard.html — BFCL V4 Leaderboard
[2]	gorilla.cs.berkeley.edu/blogs/15_bfcl_v4_web_search.html — BFCL V4 Agentic Web Search blog
[3]	huggingface.co/datasets/gorilla-llm/Berkeley-Function-Calling-Leaderboard — BFCL dataset card
[4]	github.com/ggml-org/llama.cpp/blob/master/docs/autoparser.md — llama.cpp autoparser architecture
[5]	github.com/ShishirPatil/gorilla — Gorilla repo (BFCL, Apache 2.0)
[6]	github.com/drwjirkpatrick-web/bfcl-jetson-test — BFCL Jetson Test v1 (our repo)
[7]	gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html — BFCL V3 multi-turn blog
[8]	github.com/ggml-org/llama.cpp — llama.cpp repo